Optimus-Cost-Agent

Test Strategy

Validation Plan - Phase 1, Sprint 1 - Gateway-Centric Local Runtime

Version 1.5

Architected by: Vibhanshu Agarwal

1. Test objectives

The strategy proves that a developer can complete full Plan and Agent runs with only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY` in the agent process. No upstream credential is resolvable by the agent. Separately, the loopback Gateway receives exactly its approved aggregator credential and never exposes it.

2. Scope and non-scope

In scope:

- strict loopback URL and bind enforcement, including WSL2 same-network-namespace topology;
- OpenRouter default and bounded Vercel option through the OpenAI-compatible transport;
- deterministic search live compatibility and plugin deprecation gate;
- include/exclude domain enforcement and returned-URL revalidation;
- independent package/OSV routes without search configuration;
- bounded extract provenance, redirect, SSRF, media, size, and time controls;
- provider-reported usage/cost authority and full request attribution;
- OTel/OTLP export to a real Phoenix tier;
- separate agent and Gateway credential/egress scans.

Out of scope:

- hosted staging Gateway, OAuth/device flow, tenants, org/project wallets, Vault;
- direct provider adapters;
- cross-run spend policy (`P9.85-FU-3`);
- MCP Gateway endpoint or contract;
- engine/model comparison matrices.

3. Test pyramid and evidence tiers

Tier	Dependency	Fakes permitted	Claim supported
Unit	In-process functions	Yes	Local logic and contracts
Contract	Recorded request/response shape	Yes	Parser and schema behavior
`requires_redis`	Real TimeSeries-capable Redis	No	Persistence and retention
`requires_gateway`	Real loopback Gateway + approved credential	No	Gateway/provider behavior
ACP protocol	Independent `acpx` client	No	Real client compatibility
E2E	Spawned ACP process + named dependencies	No	Golden workflow
Release	Full process/credential/ egress evidence	No	Phase 1 sign-off

There is no hosted staging Gateway. Provider fakes remain confined to unit and contract tests and cannot justify a live claim.

Verification ownership

Every design claim maps to an executable unit, integration, E2E, eval, or release-gate check. Coverage remains at least 80% aggregate Python production code, and safety-critical modules do not regress.

LLD Section	Design Claim	Test Category (§)	Tier
§4A — State Machine	AwaitingApproval gate cannot be bypassed	§6 — Mode/State	Unit + Integration
§6 — Gateway Layer	POST /v1/responses uses input field, not messages	§8 — Gateway	Unit + Integration
§6 — Gateway Layer	No provider key present in local environment during run	§8 — Gateway	E2E (Release Gate)
§9 — Tool Policy	EvidenceRequest.query sent verbatim (not substituted)	§7 — Tools	Unit
§9 — Tool Policy	EvidenceExtractRequest rejects out-of-set URLs	§7 — Tools	Unit
§9 — Evidence Ledger	total_cost_usd() reconciles against gateway response	§9 — Cost	Unit + Integration
§9D — Gateway Policy	Gateway revalidates domain allowlist independently	§8 — Gateway	Integration
§10 — Telemetry	RedisTimeSeries TS.CREATE idempotent with TS.ALTER fallback	§10 — Telemetry	Integration
§11 — Sprint 1 Gate	One-key setup: full run with only OPTIMUS_* env vars	§12 — Release Gate	E2E

5. Mode and State Transition Tests

LLD reference: §4 Behavioral Governance, §4A Agent State Model. These tests prove the state machine is deterministic and that no path bypasses the mode enforcement primitives.

Unit Tests — assert_mutation_allowed()

- ☐ In Plan/Chat mode: calling assert_mutation_allowed() raises Code -32002 with message "mutation forbidden in Plan/Chat mode".
- ☐ In Agent mode before AwaitingApproval: assert_mutation_allowed() raises Code -32002.
- ☐ In Agent mode after approval granted: assert_mutation_allowed() returns None (no exception).
- ☐ Calling any mutation tool (write_file, shell_exec, shadow_apply) in Plan/Chat mode raises -32002 before any I/O occurs.

Unit Tests — State Transitions

- ☐ Idle → Planning: valid on any user request.
- ☐ PlanReady → Executing (direct): rejected with -32002; must pass through AwaitingApproval.
- ☐ AwaitingApproval → ChatOnly on timeout: confirmed after configurable timeout_ms expires.
- ☐ Failed → Planning: retry counter increments; failure context injected into next prompt payload.
- ☐ Failed → Terminated: after max_retries=3 consecutive gate failures, state transitions to Terminated and user_escalation signal emitted.

Integration Tests — Mode Boundary

- ☐ Full Plan/Chat run: agent completes discussion and returns plan text; no file in working tree is modified; telemetry records 0 mutation calls.

6. Tool Invocation Tests

- Search is authorized only by existing policy signals and the harness gate.
- The Gateway forwards the effective include/exclude policy and rejects off-policy citation URLs.
- At `max_tokens=16`, the live deterministic plugin returns typed URL annotations or fails.
- URL, title, and non-empty content are required; annotation offsets are ignored.
- Extract requires exact prior Gateway-recorded provenance and passes redirect/SSRF/media/size/time bounds.
- Package and OSV routes remain callable without a Tavily/search credential.
- Search, extract, package, and advisory usage envelopes reconcile independently.

7. OptimusGatewaySettings Unit Tests

- Default `http://127.0.0.1:8765` passes.
- `localhost` and canonical IPv6 loopback pass.
- Non-loopback DNS names and IP literals fail closed.
- URI userinfo, ambiguous host parsing, and non-HTTP(S) schemes fail closed.
- No field or environment variable authorizes non-loopback.
- `production_mode`, extra origins, hosted origins, and signed tenant profiles are absent.
- `optimus_api_key` is masked in repr, serialization, logs, telemetry, and state.

7.1 Integration, E2E, and egress gates

- Direct policy-violation tests target the real loopback Gateway.
- Aggregator routing/fallback evidence records the returned provider and model.
- The agent and ACP child may egress only to the loopback Gateway.
- The Gateway may egress only to the configured aggregator, approved package/OSV hosts, approved extract targets, Redis, and configured OTLP endpoint.
- Agent and Gateway environments are scanned separately.
- The agent has no upstream key; the Gateway has the approved aggregator key by design.
- After replacement acceptance, the Gateway has no direct-provider, Tavily, or LangSmith key.
- Crafted tool output cannot cause direct agent egress.
- WSL2 evidence proves both processes share one network namespace.

Failure behavior

Malformed or missing provider usage/cost fails closed. Permanent authentication, policy, schema, and validation failures do not retry. Transient faults have a bounded retry budget and record every attempt.

8. Cost Accounting Tests

LLD reference: §10 Usage Accounting Service, §9 Evidence Ledger. These tests prove that cost accounting is accurate, complete, and reconciles against the gateway response.

Unit Tests — EvidenceLedger

- ☐ total_cost_usd() sums cost_usd across all EvidenceLedgerEntry objects; returns 0.0 on empty ledger.
- ☐ total_billing_units() sums billing_units; matches provider-native unit count.
- ☐ total_credits() (backward compat) sums credits_used; does not conflict with cost_usd total.
- ☐ GatewayUsage fields (gateway_request_id, provider, provider_request_id, cache_hit, billing_units, cost_usd) all propagate correctly from gateway response envelope to EvidenceLedgerEntry.
- ☐ Ledger entries are append-only; no existing entry can be modified after record() is called.

Unit Tests — Pricing Fallback

- ☐ Missing pricing_snapshot.json: usage accounting engine falls back to default pricing matrix and raises telemetry audit signal PRICING_FALLBACK_ACTIVATED.
- ☐ pricing_snapshot.json present but stale (> 24h): PRICING_SNAPSHOT_STALE audit signal raised; calculation proceeds with stale data.

Integration Tests — RedisTimeSeries

- ☐ TS.CREATE with retention 30 days: key created; RETENTION verified via TS.INFO.
- ☐ Duplicate TS.CREATE on existing key: TS.ALTER applied idempotently; RETENTION updated; no error thrown.
- ☐ TS.ADD with run_id tag: confirmed present in TS.RANGE query with tag filter.
- ☐ Run metadata hash (HSET): execution_mode, generation_scope, rigor_level, assumption ledger count all written at workflow completion; TTL set to 30 days.

8A. Test Coverage Target, Measurement & Trace Observability

> This document is the authoritative source of truth for the Phase 1 coverage target, the measurement-tool taxonomy, and trace observability. The HLD (§12, §11A) and LLD (§11A) may **summarize** the target for context, but Test Strategy §8A remains authoritative; where any summary diverges, this section governs.

Test Coverage Target and Measurement

Phase 1 targets a minimum **80% aggregate Python test coverage** threshold for production code, measured in CI with coverage.py and surfaced through pytest-cov during normal pytest execution. The 80% threshold is a release gate, not a quality substitute: deterministic safety-critical modules such as ToolRegistry authorization, MutationGuard, GatewayUsage accounting, EvidenceLedger reconciliation, JSON-RPC framing, retry/fail-closed logic, and policy enforcement should trend materially higher and must not regress without explicit review.

Coverage.py is the canonical coverage engine for line and branch coverage reporting; pytest-cov is the preferred pytest integration for local and CI workflows. Agent-quality and AI-safety evaluations

8A. Observability Test Strategy

Tool	Category	Purpose	Counts toward coverage
`coverage.py` / `pytest-cov`	Code coverage	Production-code execution	Yes
DeepEval / Ragas	LLM quality	Correctness and groundedness	No
PyRIT	Adversarial	Injection and tool-control safety	No
OpenTelemetry / OTLP	Trace observability	Vendor-neutral span contract and export	No

Run a real Phoenix evidence tier and assert:

- required span attributes and request correlation;
- parent/child relationships;
- secret redaction;
- batching and retry outcome;
- validation and policy disposition;
- final failure/success disposition.

Phoenix is the documented default, not an API dependency. No LangSmith dependency, endpoint, credential, or amortized per-request charge exists.

7A. Deterministic Search Compatibility Gate

The deterministic search compatibility gate uses one cheap Flash/Haiku-class model and one default engine request shape. The approved 2026-07-27 baseline is:

- `max_tokens=16`;
- 3 annotations on the minimal-output probe;
- 3/3 include-domain results allowed and 0 violations;
- 0/3 excluded-domain results forbidden and 0 violations;
- 9/9 annotations with HTTPS URL, title, and non-empty content;
- mean latency 7,068.7 ms/search;
- provider-reported mean cost `$0.0051584/search`;
- direct extract 270,008 bytes -> 60,077 text characters in 589.5 ms.

These are evidence values, not permanent performance thresholds. Each run reports fresh values and fails on missing search execution, annotations, domain enforcement, or provider usage/cost. Engine and model comparison matrices are out of scope. A verified-or-fail server-tool successor requires a separate live test proving `max_uses: 1`, exactly one search, search-use accounting, valid annotations, domain enforcement, and fail-closed behavior when execution evidence is absent.

9. Error, Retry, and Failure Injection Tests

LLD reference: §6 Resilient Provider Calling Layer, §4A Failure and Retry Lifecycle. These tests prove that transient failures are retried with backoff and permanent failures abort cleanly without side effects.

Unit Tests - RetryPolicy

- `TransientGatewayError` on first attempt: tenacity retries up to `max_retries=3`; succeeds on 3rd attempt; `retry_count=2` in metadata.
- `ProviderRateLimitError`: exponential backoff applied (500 ms base); jitter present.
- `PermanentGatewayError`: `ABORT_WITH_REPORT` returned immediately; no retry; failure report emitted.
- `PolicyViolationError`: `ABORT_WITH_REPORT`; escalation signal emitted.
- `BudgetExhaustedError`: `ABORT_WITH_REPORT`; run terminates; `cost_usd` at time of abort recorded in ledger.
- `max_retries=3` exceeded: `ESCALATE_TO_USER` returned; `prior_failures` injected into user escalation payload.

Integration Tests - Failure Injection

- Gateway returns 503 twice, then 200: agent retries twice, succeeds; working tree unchanged after failed attempts; final patch applied only on success.
- Fitness gate fails on attempts 1 and 2, passes on attempt 3: `replan_context()` is injected with failure summaries on attempts 2 and 3; final patch differs from attempt 1.
- Budget exhausted mid-run: run terminates gracefully; partial telemetry is flushed; no partial file writes occur in the working tree.

10. Schema Validation Tests

LLD reference: §1 ACP Protocol Framing, §9 Tool Registry. These tests prove malformed inputs are rejected before processing and Pydantic v2 validators enforce invariants at the boundary.

Unit Tests - ACP Framing

- `Content-Length: 0` with non-empty body: rejected with `-32700 Parse Error`.
- `Content-Length > MAX_HEADER_SIZE`: rejected with `-32600 Invalid Request`.
- Non-numeric `Content-Length`: rejected with `-32700 Parse Error`.
- Body larger than declared `Content-Length`: truncated; remaining bytes are not leaked into the next message.
- JSON-RPC body missing `method`: rejected with `-32600 Invalid Request`.
- JSON-RPC body with unknown method: rejected with `-32601 Method Not Found`.

Unit Tests - Pydantic Models

- `EvidenceRequest` with an empty query string: `ValidationError` before any Gateway call.
- `EvidenceExtractRequest` with `max_chars_per_source=0`: `ValidationError`.
- `OptimusGatewaySettings` with empty `optimus_api_key`: `ValidationError`.
- `GatewayUsage` with negative `billing_units`: `ValidationError`.

11. Security and Trust Boundary Tests

- Secret scans include the aggregator key and every retired direct-provider, Tavily, and LangSmith key name.
- Strict-loopback validation replaces production-mode permutations.
- URL parsing rejects userinfo, non-HTTP(S), non-loopback hosts, ambiguous IP forms, and DNS rebinding seams.
- Extract revalidates every redirect and final address and blocks private/link-local/loopback resolution.
- Tool output is untrusted and cannot become policy or executable content.
- Provider-reported cost must be Decimal-parseable, non-negative, and attributable.
- Logs and traces preserve field names while redacting secrets.

Golden task evidence

Golden tasks run against the real local Gateway. Named live claims use real dependencies. ACP protocol evidence uses `acpx`, not a project-authored client.

12. Golden Task Regression Suite

Every golden task records:

- expected mode and final state;
- expected tool class and authorization outcome;
- cost band and provider-reported usage;
- mutation behavior;
- Gateway and provider request identity;
- trace identity and final disposition.

The suite covers Plan-only refusal, approved Agent mutation, deterministic search, bounded extract, free package/advisory routes, retry exhaustion, malformed usage/cost, and credential leakage attempts.

OTel/OTLP-to-Phoenix evidence replaces LangSmith assertions. A trace failure cannot silently count as success; delivery state and final disposition must be explicit.

Process-scope assertion

The agent has no upstream credential. The Gateway has exactly the approved aggregator credential and never exposes it through logs, telemetry, state, responses, child environments, or error text.

13. Phase 1 Release Gates

Transport and protocol

- Shadow workspace, mutation guard, and independent ACP-client evidence pass.
- Strict-loopback topology and WSL2 namespace evidence pass.
- Both agent-facing completion shapes validate; mixed shapes fail.

Runtime and credentials

- Full Plan and Agent runs use only the two agent-facing variables.
- Agent and child processes have no direct external egress.
- Gateway has one approved aggregator credential and no retired keys after acceptance.

Evidence and cost

- Search plugin compatibility/deprecation gate passes.
- Package and OSV routes work without search configuration.
- Extract provenance and SSRF gates pass.
- Provider-reported usage/cost reconciles across response, ledger, and TimeSeries.

Observability

- Real OTLP spans reach Phoenix with required fields and redaction.
- No LangSmith dependency or amortized charge exists.

Release sign-off remains governed by the authoritative Plan 9.6 live-verification gate.

14. Guardrail & Workflow Test Cases

LLD reference: §12 (Guardrail & Workflow Component Contracts). Companion reference: **Agent Execution Guardrails & Workflow Strategy v1.0** §11. Consistent with Objective 1 (§1), each control below traces to at least one executable test category; a control without a category is not considered validated for Phase 1.

14.1 Permission policy tests — deny precedence over allow; mode short-circuit; impact-class `HOLD`; classifier cannot overturn a deny.

14.2 Shell validator tests — destructive, pipe-to-shell, environment/credential access, ANSI, insecure-transport, and egress patterns all BLOCK before any subprocess is spawned.

14.3 Unicode / homoglyph tests — Cyrillic-vs-Latin confusables in hostnames and paths (e.g. U+0456 vs U+0069) are detected and held/blocked.

14.4 Prompt-injection fixture tests — poisoned agent config and poisoned MCP tool metadata are caught by `ConfigTrustScanner` / `MCPTrustRegistry` on ingest.

14.5 MCP autoload denial tests — a server bundled in a cloned repo does not auto-load; a manifest-hash change forces re-approval; `allowed_tools` are enforced.

14.6 Pre-commit / CI parity tests — the same rule set (Ruff, Bandit, AST-grep, hygiene hooks) fails identically locally and in a clean CI checkout.

14.7 Bypass tests — `--no-verify`, force-push to `main`, unsafe `.env` reads, and unsafe network commands are all blocked.

14.8 Loop control tests — the loop stops on completion, on `max_iterations`, on budget exhaustion, and on repeated-failure detection; it never bypasses §14.1/§14.2 enforcement.

14.9 Skill loading & trust tests — a skill loads only when matched; an untrusted/draft skill is blocked; declared `allowed_tools` are enforced; a skill cannot override project/user deny rules.

14.10 Additions to the §4 Requirements-to-Test Traceability Matrix

LLD Section	Design Claim	Test Category (§)	Tier
§12A — Permission	Deny precedes allow; classifier cannot overturn deny	§14 — Guardrails	Unit
§12A — Pre-Tool Guard	Bad tool call blocked before execution	§14 — Guardrails	Unit + Integration
§12A — Shell Validator	Homoglyph / pipe-to-shell / egress blocked pre-spawn	§14 — Guardrails	Unit
§12B — MCP Trust	No auto-load from cloned repo; hash change re-approves	§14 — Guardrails	Unit + Integration
§12B — Config Scan	Poisoned config / tool metadata caught on ingest	§14 — Guardrails	Unit
§6 / §12 — CI Parity	Local and clean-CI checks fail identically	§14 — Guardrails	Integration

§12C — Goal Loop	Stops on completion / max-iter / budget / repeated failure	§14 — Guardrails	Unit + Integration
§12D — Skills	Match-only load; draft blocked; allowed-tools enforced	§14 — Guardrails	Unit

Release-gate note: §13 (Phase 1 Release Gates) is extended with a release-blocking "no guardrail bypass" check covering §14.7 above.